

计算机设计与实践

流水线CPU及SoC设计

任务A：控制冒险、冒险检测

2026·夏



HITSZ 实验与创新实践教育中心
Education Center of Experiments and Innovations, HITSZ

实验目的

- ◆ 理解流水线冒险检测原理
- ◆ 理解静态分支预测解决控制冒险的原理
- ◆ 掌握冒险检测、静态分支预测的实现方法



实验内容

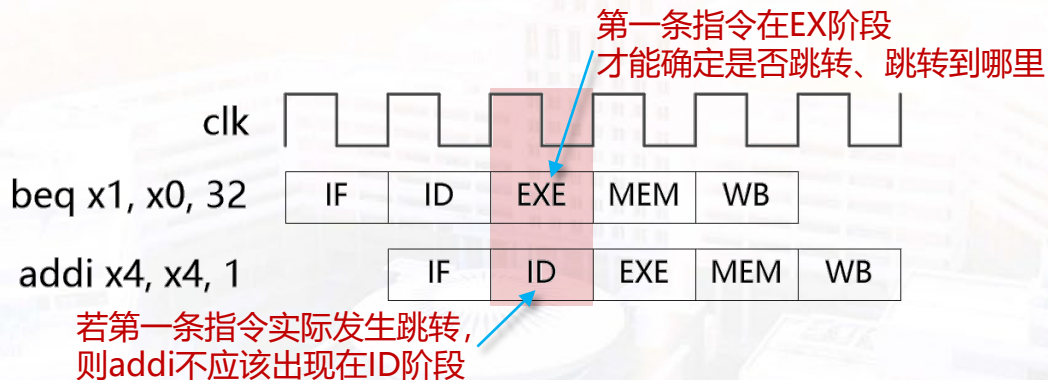
- ◆ 添加冒险检测模块，实现控制冒险、数据冒险的检测逻辑
- ◆ 在理想流水线数据通路中实例化冒险检测模块，并连接相应信号
- ◆ 增加处理控制冒险所需的流水级清空逻辑
- ◆ 对冒险检测、控制冒险处理进行功能仿真验证



流水线冒险检测

◆ 控制冒险

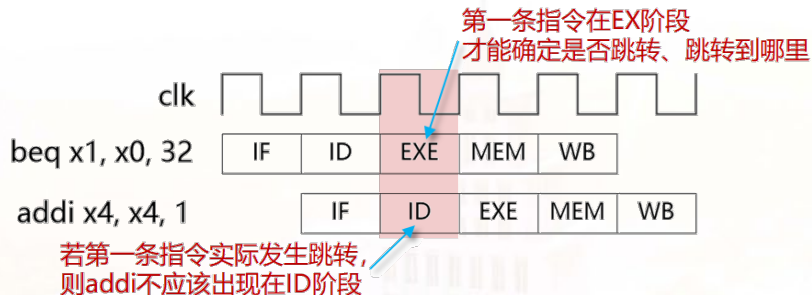
- 一条指令是否应该执行，依赖于前面一条尚在流水线中的指令
- 因**无法确定取指方向**，导致流水线中可能出现本不该执行的指令



- 检测EX阶段指令是否跳转 —— 是直接跳转指令？条件分支的条件判断为真？
- EX阶段指令发生跳转时，**清除ID阶段**，**IF阶段使用跳转目标地址**重新取指

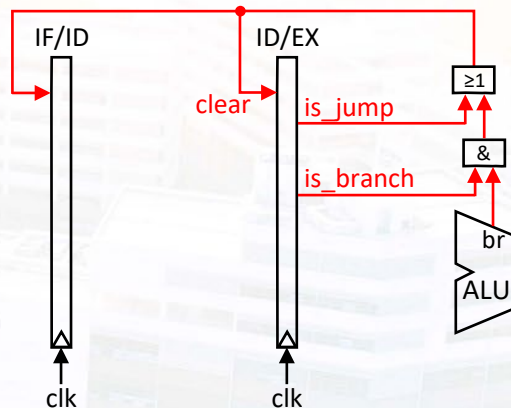
控制冒险处理

◆ 解决控制冒险



◆ 实现方法

- ① 在EX阶段检测指令是否发生跳转
- ② 清除ID阶段: 清空IF/ID、ID/EX
- ③ 修正PC, 同时使用跳转目标地址重新取指



控制冒险处理

- 清空流水线寄存器

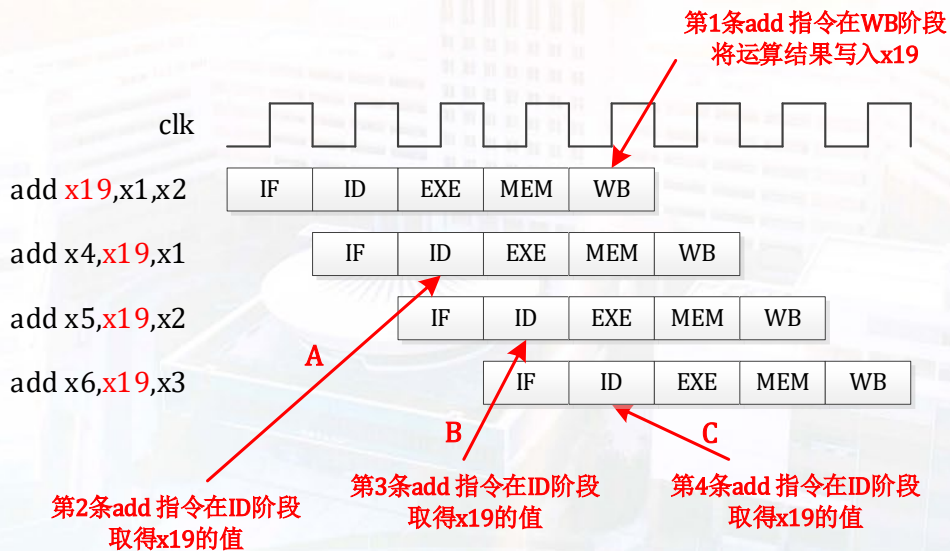
RTL实现示例 (ID/EX)

```
01 always @ (posedge clk or posedge rst) begin
02     if (rst)          ex_valid <= 1'b0;
03     else if (clear) ex_valid <= 1'b0;
04     else              ex_valid <= id_valid;
05 end
06
07 always @ (posedge clk or posedge rst) begin
08     if (rst)          ex_rf_we <= 1'b0;
09     else if (clear) ex_rf_we <= 1'b0;
10     else              ex_rf_we <= id_rf_we;
11 end
```

流水线冒险检测

◆ 数据冒险

一条指令依赖于前面一条尚在流水线中的指令；因**无法提供指令执行所需数据**而导致指令不能在预期的时钟周期内执行的情况



流水线冒险检测

◆ 数据冒险检测

情形A:

ID/EX.RD = ID.RS1 = x19

ID/EX.RD = ID.RS2 = x19

情形B:

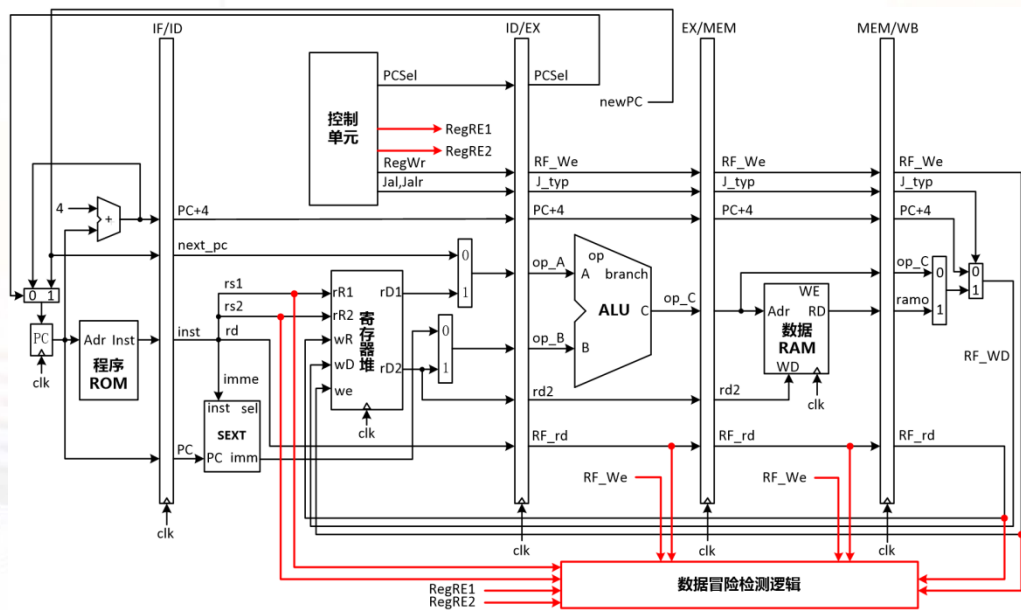
EX/MEM.RD = ID.RS1 = x19

EX/MEM.RD = ID.RS2 = x19

情形C:

MEM/MB.RD = ID.RS1 = x19

MEM/WB.RD = ID.RS2 = x19



当前ID阶段，寄存器是否有被读取 排除0号寄存器

RTL实现

```
1 wire rs1_id_wb_hazard = (wb_rd == id_rs1) & wb_we & id_rf1 & (wb_rd != 5'h0);  
2 wire rs2_id_wb_hazard = (wb_rd == id_rs2) & wb_we & id_rf2 & (wb_rd != 5'h0);
```


仿真验证

- ◆ 在Vivado中使用默认的lw.coe程序运行功能仿真
- ◆ 把控制冒险检测、流水线清空的相关信号添加到波形，观察波形，分析lw.coe中的直接跳转指令、条件分支指令能否正确执行
- ◆ 把数据冒险检测的相关信号添加到波形，观察波形，分析发生RAW冒险时，能否正确检测到数据冒险
- ◆ **注意代码规范性，可以避免很多问题！**



HITSZ 实验与创新实践教育中心
Education Center of Experiments and Innovations, HITSZ

